

Timer Interrupt

Chao-Chin Wu

Reference: M. Margolis, et al., Arduino Cookbook, 3rd Edition,
Oreilly, 2020

https://electronoobs.com/eng_arduino_tut140.php

Interrupt

- An interrupt is a signal to the processor emitted by hardware or software indicating an event that **needs immediate attention**.
- Whenever an interrupt occurs, the controller completes the execution of the current **instruction** and starts the execution of an **Interrupt Service Routine (ISR)** or **Interrupt Handler**.
- ISR tells the processor or controller what to do when the interrupt occurs.
- The interrupts can be either **hardware interrupts** or **software interrupts**.

Hardware Interrupt

- A hardware interrupt is an **electronic alerting signal** sent to the processor from an external device
 - like a disk controller or an external peripheral.
- For example, when we press a key on the keyboard or move the mouse, they trigger hardware interrupts which cause the processor to read the keystroke or mouse position.

Software Interrupt

- A software interrupt is caused either by an **exceptional condition** or a **special instruction** in the instruction set
 - which causes an interrupt when it is executed by the processor.
- a divide-by-zero exception
 - causing the computer to abandon the calculation or display an error message.
- Software interrupt instructions work similar to subroutine calls.

Interrupt Service Routine

- For every interrupt, there must be an interrupt service routine (**ISR**), or **interrupt handler**.
- When an interrupt occurs, the microcontroller runs the interrupt service routine.
- For every interrupt, there is a **fixed location** in memory that holds the address of its interrupt service routine, **ISR**.
- The table of memory locations set aside to hold the addresses of ISRs is called as the **Interrupt Vector Table**.

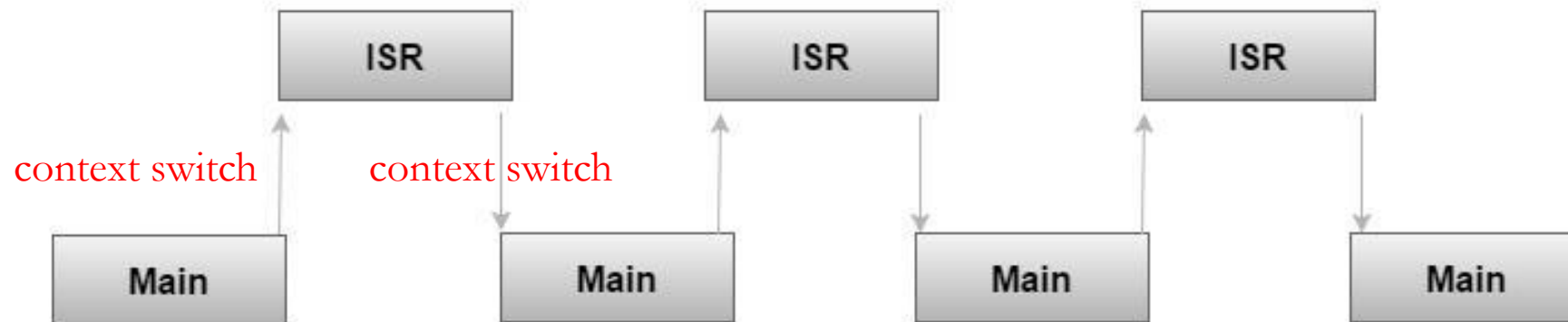
Program Execution without Interrupts

Time →



Program Execution with Interrupts

Time →



ISR : Interrupt Service Routine

Exception number	IRQ number	Offset	Vector
16+n	n	0x0040+4n	IRQn
.	.	.	.
.	.	.	.
.	.	.	.
18	2	0x004C	IRQ2
17	1	0x0048	IRQ1
16	0	0x0044	IRQ0
15	-1	0x0040	Systick
14	-2	0x003C	PendSV
13		0x0038	Reserved
12			Reserved for Debug
11	-5	0x002C	SVCall
10			
9			
8			Reserved
7			
6	-10	0x0018	Usage fault
5	-11	0x0014	Bus fault
4	-12	0x0010	Memory management fault
3	-13	0x000C	Hard fault
2	-14	0x0008	NMI
1		0x0004	Reset
		0x0000	Initial SP value

starting address of IRQ2_ISR

```
IRQ2_ISR ()
{
.....
.....
.....
}
```

starting address of BusFault_ISR

```
BusFault_ISR ()
{
.....
}
```

- In general, the vector table is fixed at address 0x00000000 on **system reset**.
- Once an exception or interrupt is triggered, the processor **automatically jumps** to the corresponding address in the vector table
 - which contains **an address** to the relevant exception or interrupt handlers (**ISR**).

Steps to Execute an Interrupt

- The microcontroller closes the currently executing instruction and saves the address of the next instruction (PC) on the stack.
- It also saves the current status of all the interrupts internally (i.e., not on the stack).
- It jumps to the memory location of the interrupt vector table that holds the address of the interrupts service routine.

Steps to Execute an Interrupt

- The microcontroller gets the address of the ISR from the interrupt vector table and jumps to it.
- It starts to execute the interrupt service subroutine.
- the microcontroller returns to the location where it was interrupted.
 - First, it gets the program counter (PC) address from the stack by popping the top bytes of the stack into the PC.
 - Then, it start to execute from that address.

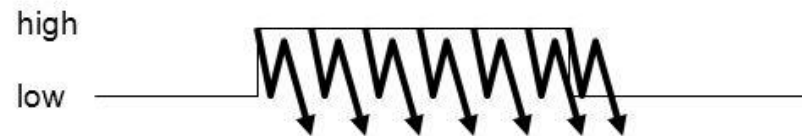


Edge- vs. Level-Triggered Interrupts

Edge triggered



Level triggered



- Edge triggered interrupts signal as a **one shot** event!
- Level triggered interrupts are signaled **as long as line is raised**

- Level triggered

- as long as the IRQ line is asserted, you get an interrupt request.
- When you serve the interrupt and return, if the IRQ line is still asserted, you get the interrupt again immediately.

- Edge triggered

- You get an interrupt when the line changes from inactive to active state, but only once.
- To get a new request, the line must go back to inactive and then to active again.

Enabling and Disabling an Interrupt

- Upon Reset, all the interrupts are disabled even if they are activated.
- The interrupts must be enabled using software in order for the microcontroller to respond to those interrupts.

Interrupt Priority

- Some systems allow to alter the interrupt priority by assigning the higher priority to any one of the interrupts.

Interrupt inside Interrupt

- a high-priority interrupt can interrupt a low-priority interrupt
- a low-priority interrupt can be interrupted by a high-priority interrupt, but not by any another low-priority interrupt

About Interrupt Service Routines on Arduino

- An ISR **cannot** have any **parameters**, and they **shouldn't return anything**.
- an ISR should be as **short and fast** as possible.
- If your sketch uses multiple ISRs, only one can run at a time
 - other interrupts will be executed after the current one finishes in an order that depends on the **priority** they have.

-
- **Serial data** received while in the function may be **lost**.
 - **millis()** relies on interrupts to count, so it will never increment inside an ISR.
 - Since **delay()** requires interrupts to work, it will not work if called inside an ISR.
 - **micros()** works initially but will start behaving erratically after 1-2 ms.
 - **delayMicroseconds()** does not use any counter, so it will work as normal.

-
- Typically **global variables** are used to pass data between an ISR and the main program.
 - To make sure variables shared between an ISR and the main program are updated correctly, declare them as **volatile**.
 - **volatile variable**
 - a variable that is marked or cast with the keyword "volatile"
 - the variable can be changed by some outside factor, such as the operating system or other software. (<https://www.techopedia.com/definition/22856/volatile-variable>)

Volatile Variables

- In computer programming, particularly in the C, C++, C#, and Java programming languages, the **volatile keyword** indicates that a value may change **between** different accesses, even if it does not appear to be modified.
- This keyword prevents an **optimizing compiler** from optimizing away subsequent reads or writes and thus incorrectly **reusing a stale value** or **omitting writes**.
- Volatile values primarily arise in **hardware access** (memory-mapped I/O),
 - reading from or writing to memory is used to communicate with **peripheral devices**,
 - in **threading**, where a different thread may have modified a value.

- Example of memory-mapped I/O in C

```
static int foo;

void bar(void) {
    foo = 0;

    while (foo != 255);
}
```

An optimizing compiler will notice:

- no other code can possibly change the value stored in `foo`
- assume that `foo` will remain equal to 0 at all times.



Optimization

```
void bar_optimized(void) {
    foo = 0;

    while (true);
}
```

Volatile Variables

- the behavior of volatile **differs** significantly between programming languages, and is easily misunderstood.
- In C and C++
 - it is a **type qualifier**, like const, and is a property of the type.
 - it **does not work** in most **threading** scenarios, and that use is discouraged.
- In Java and C#,
 - it is a **property of a variable**
 - the object (to which the variable is bound) may **mutate**, and is specifically intended for **threading**.

attachInterrupt() in Arduino

- **Digital Pins With Interrupts**
- The first parameter to attachInterrupt() is an **interrupt number**.
 - Normally you should use **digitalPinToInterrupt(pin)** to translate the actual digital pin to the specific interrupt number.
 - set digitalPinToInterrupt(3) if you connect to pin 3

BOARD	DIGITAL PINS USABLE FOR INTERRUPTS
Uno, Nano, Mini, other 328-based	2, 3

Syntax

- attachInterrupt(**digitalPinToInterrupt(pin)**, ISR, mode) (**recommended**)
- attachInterrupt(interrupt, ISR, mode) (not recommended)
- attachInterrupt(pin, ISR, mode) (Not recommended).

Parameters

- **interrupt**: the number of the interrupt. Allowed data types: int.
- **pin**: the Arduino pin number.
- **ISR**: the ISR to call when the interrupt occurs
- **mode**: defines when the interrupt should be triggered.

-
- Four **Mode constants** are predefined as valid values:
 - **LOW** to trigger the interrupt whenever the pin is low,
 - **CHANGE** to trigger the interrupt whenever the pin changes value
 - **RISING** to trigger when the pin goes from low to high,
 - **FALLING** for when the pin goes from high to low.
 - The Due, Zero and MKR1000 boards allow also:
 - **HIGH** to trigger the interrupt whenever the pin is high.

Interrupt Numbers

- Normally you should use `digitalPinToInterrupt(pin)`,
 - rather than place an interrupt number directly into your sketch.
 - compatibility trouble

Example Code

```
const byte ledPin = 13;  
const byte interruptPin = 2;  
volatile byte state = LOW;
```

// pin 2狀態改變時，產生中斷

```
void setup() {  
  pinMode(ledPin, OUTPUT);  
  pinMode(interruptPin, INPUT_PULLUP);  
  attachInterrupt(digitalPinToInterrupt(interruptPin), blink, CHANGE);  
}
```

```
void loop() {  
  digitalWrite(ledPin, state);  
}
```

// 在loop()中不會調用blink()
// 在整個程式中不會直接調用blink()

```
void blink() { //ISR  
  state = !state;  
}
```

// 中斷發生時才執行blink()
// blink()中變更全域變數state，需要宣告成volatile

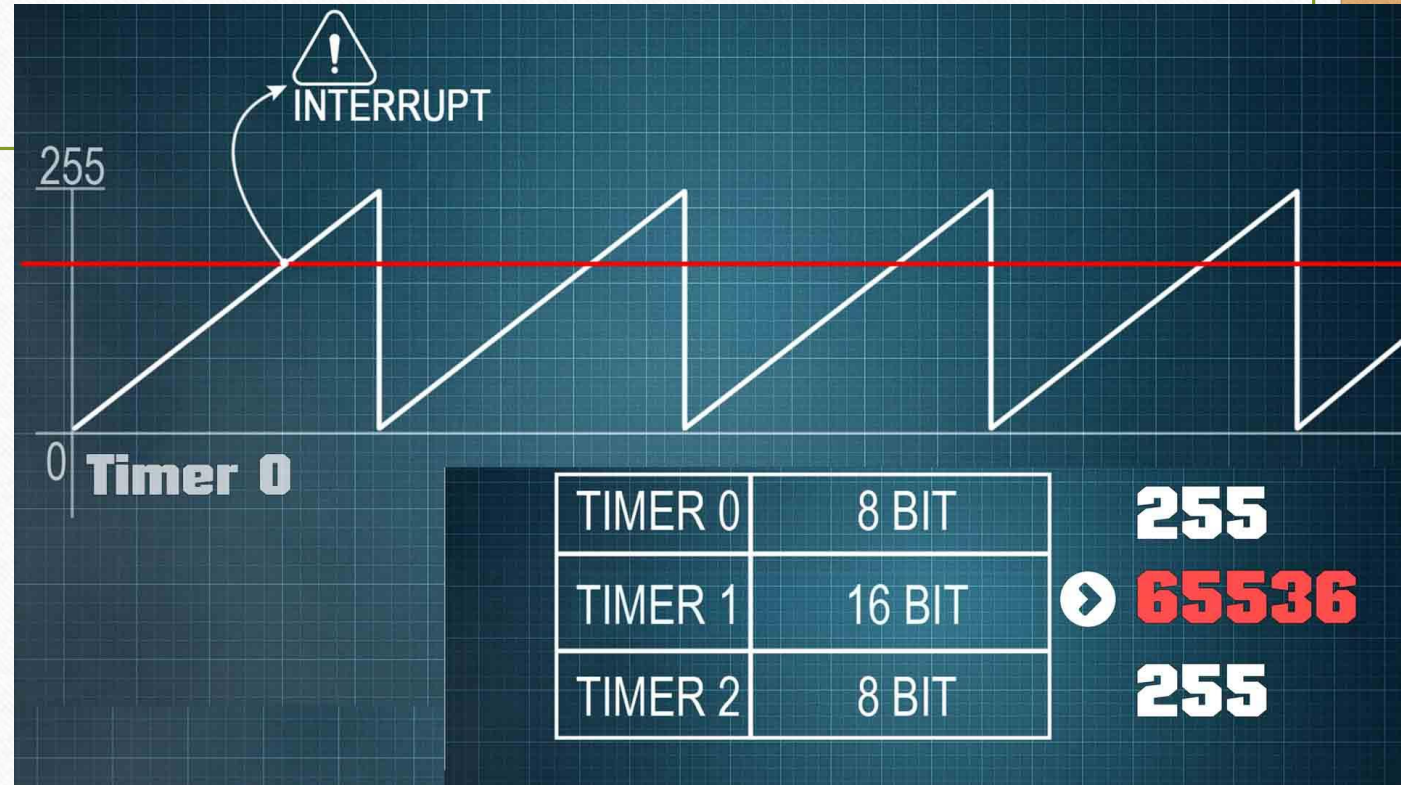
Arduino Timer

- A timer is a piece of hardware **built in** the Arduino controller
- the Arduino UNO has **3 timers**, **Timer0**, **Timer1** and **Timer2**.
 - like a clock, and can be used to measure time events.
- The timer can be **programmed** by some special registers

- **TCCR_x** - Timer/Counter Control Register.
 - The **prescaler** can be configured here.
- **TCNT_x** - Timer/Counter Register.
 - The **actual timer value** is stored here.

- **OCR_x** - Output Compare Register
 - When to have a **time-up**
- **ICR_x** - Input Capture Register
 - (only for **16bit** timer)
- **TIMSK_x** - Timer/Counter Interrupt Mask Register.
 - To **enable/disable** timer interrupts.
- **TIFR_x** - Timer/Counter Interrupt Flag Register.
 - Indicates a **pending** timer interrupt.

- timer 0 and 2 are 8 bits
 - could count from 0 to 255.
- Timer 1 is 16 bits
 - could count from 0 to 65535.
- Once a timer reaches its maximum value, it will go back to 0 or start decreasing depending on the selected mode.
- Timer will create a **triangle shape curve** and we could use this to create our **interruptions**.



Timer ISR Modes

- **compare match**
 - when the timer value is equal to the compare value, it will trigger the interrupt
- **overflow**
 - an interruption is triggered each time the timer overflows
- **input capture interrupt**
 - the timer could store its value in a different register, each time an external event happens on one of the Arduino pins

Timer ISR Modes

- the **compare match**
 - We can write a value in a **different register**
 - when the timer value **is equal to** the compare value, it will trigger the **interrupt**.
 - For example, we set our compare match register to 100, each time timer 0 reaches 100, it will create an interruption.

Timer ISR Modes

- **overflow**
 - an interruption is triggered each time the timer overflows
- the **input capture interrupt**
 - for the Arduino UNO, could be implemented on timer 1.
 - the timer could store its value in a different register, each time an external event happens on one of the Arduino pins.

input capture function

- "Capture" mode saves the current timer value into the CCP registers when a specific event occurs (usually a transition on the CCP pin).
- used in many applications
 - Pulse width measurement
 - Period measurement
 - Capturing the time of an event

TIMER 0	Compare Match Overflow
TIMER 1	Compare Match Overflow Input Capture
TIMER 2	Compare Match Overflow

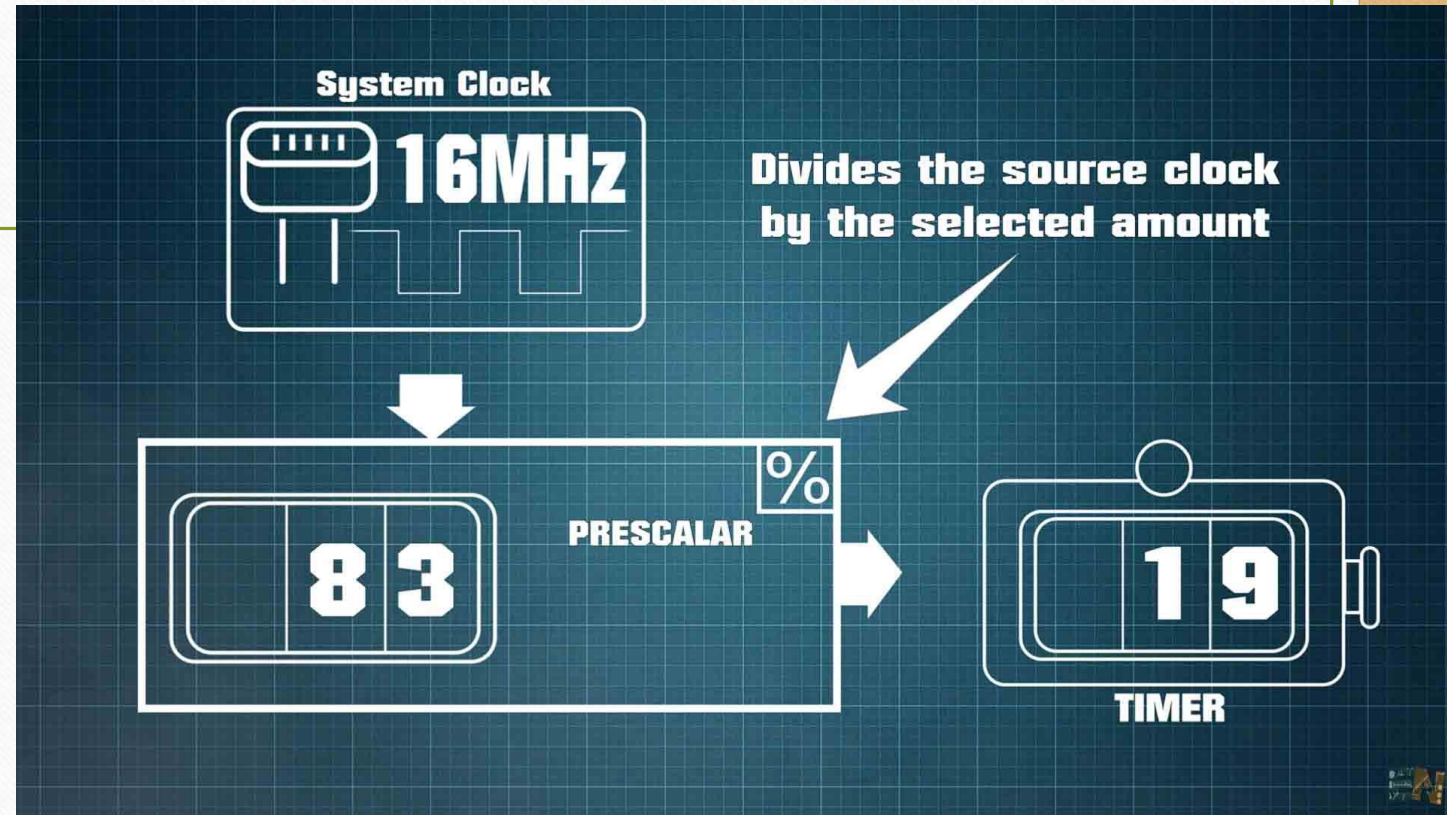
ARDUINO TIMER

- A lot of Arduino functions uses timers
 - the time functions: delay, millis, micros and delayMicroseconds.
 - the PWM analogWrite.
 - the tone and the noTone functions
 - the Servo library

The Prescaler

- Imagine a timer as a counter that counts pulses.
- This timer is fed by the system clock
 - for the Arduino UNO
 - it could be the **16Mhz crystal clock** or
 - depending on the **bootloader** or
 - an 8MHz internal oscillator.

- the main system clock is the 16Mhz crystal
 - 62.5ns for each pulse
- the **prescalar** is set to **8**
- the output signal from the prescalar is 500ns.
- So **each 8 pulses** from the system clock, our timer will **increase by 1**.



Setting the Prescaler

- First **register** we need to change is the **prescaler**.
- To control the timers we have two main **control** registers,
 - the **TCCR A** and the **TCCR B**,
 - each with the number of the timer.
 - So for timer 1 we have TCCR 1A and TCCR 1B

- Now, we need to change the **TCCRB** register.
- Here, all we care are the first 3 bits which are used to define the **prescaler** value.

15.11.2 TCCR1B – Timer/Counter1 Control Register B

Bit (0x81)	7	6	5	4	3	2	1	0	
	ICNC1	ICES1	–	WGM13	WGM12	CS12	CS11	CS10	TCCR1B
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Table 15-6. Clock Select Bit Description

CS12	CS11	CS10	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	$\text{clk}_{\text{I/O}}/1$ (no prescaling)
0	1	0	$\text{clk}_{\text{I/O}}/8$ (from prescaler)
0	1	1	$\text{clk}_{\text{I/O}}/64$ (from prescaler)
1	0	0	$\text{clk}_{\text{I/O}}/256$ (from prescaler)
1	0	1	$\text{clk}_{\text{I/O}}/1024$ (from prescaler)
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

- Using the CS10 CS11 and CS12 bits, we can
 - disable the prescaler or
 - divided by 8, 64, 256, 1024 or
 - use an external clock source.

Table 15-6. Clock Select Bit Description

CS12	CS11	CS10	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	clk _{I/O} /1 (no prescaling)
0	1	0	clk _{I/O} /8 (from prescaler)
0	1	1	clk _{I/O} /64 (from prescaler)
1	0	0	clk _{I/O} /256 (from prescaler)
1	0	1	clk _{I/O} /1024 (from prescaler)
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

Table 15-6. Clock Select Bit Description

CS12	CS11	CS10	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	clk _{I/O} /1 (no prescaling)
0	1	0	clk _{I/O} /8 (from prescaler)
0	1	1	clk _{I/O} /64 (from prescaler)
1	0	0	clk _{I/O} /256 (from prescaler)
1	0	1	clk _{I/O} /1024 (from prescaler)
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

- For the timers 0 and 2 we have to use the TCCR0B and TCCR2B and bits CS00, CS01, CS02 and bits CS20, CS21 and CS22.

Example code

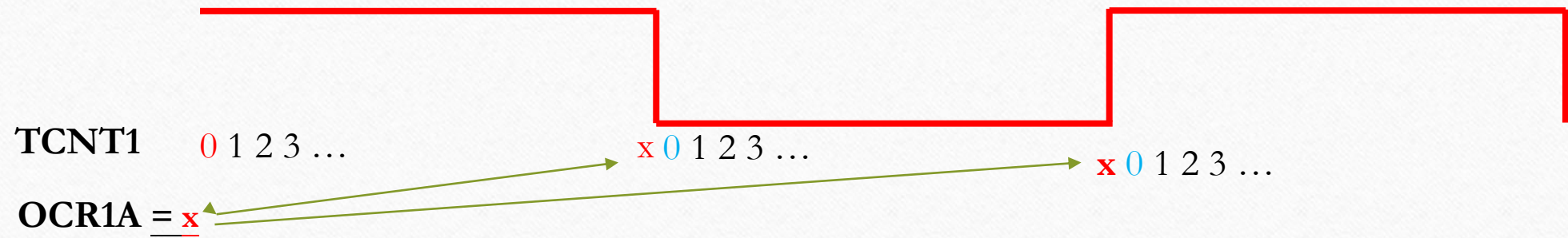
```
void setup() {  
    TCCR1A = 0;           //Reset entire TCCR1A register  
    TCCR1B = 0;           //Reset entire TCCR1B register  
    TCCR1A |= B00000100;  //Set CS12 to 1 so we get Prescaler = 256  
    TCNT1 = 0;            //Reset Timer 1 value to 0  
}                          // The timer value is stored in the TCNT register  
  
void loop() {  
    //your code here...  
}
```


Setting the Interrupt (Compare Match)

- specify **when** to make the interruption.
- To **activate** the compare match interruption we set the **TIMSK** register.
 - **setting** the bits 1 and 2, we can **enable** time compare interrupt on the value defined on **compare** registers OCRA and OCRB.
 - these OCR registers will **tell when** to make the interruption.
 - if we set OCR1A to be 2000, when timer 1 reaches 2000 it will create an interruption.

TIMSK1

BIT	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
						OCIE1B	OCIE1A	TOIE1



When $TCNT1 == OCR1A$, invert the pulse and reset the TCNT1

The **fixed** frequency is dependent on OCR1A

500ms Blink Example code

```
/* Example code with timer interrupt that will create an interruption each
 * 500ms using timer1 and prescalar of 256.
Calculations (for 500ms):
  System clock 16 Mhz and Prescalar 256;
  Timer 1 speed = 16Mhz/256 = 62.5 Khz
  Pulse time = 1/62.5 Khz = 16us
  Count up to = 500ms / 16us = 31250 (so this is the value the OCR register should have)*/
bool LED_STATE = true;
```



```
void setup() {  
    pinMode(13, OUTPUT);          //Set the pin to be OUTPUT  
    cli(); //clear interrupt, noInterrupts() //stop interrupts for till we make the settings  
    /*1. First we reset the control register to amke sure we start with everything disabled.*/  
    TCCR1A = 0;                   // Reset entire TCCR1A to 0  
    TCCR1B = 0;                   // Reset entire TCCR1B to 0  
  
    /*2. We set the prescalar to the desired value by changing the CS10 CS12 and CS12 bits. */  
    TCCR1B |= B00000100;         //Set CS12 to 1 so we get prescalar 256  
  
    /*3. We enable compare match mode on register A*/  
    TIMSK1 |= B00000010;         //Set OCIE1A to 1 so we enable compare match A  
  
    /*4. Set the value of register A to 31250*/  
    OCR1A = 31250;                //Finally we set compare register A to this value  
    sei(); //set interrupt, Interrupts() //Enable back the interrupts  
}
```

```
void loop() {
```

```
    // put your main code here, to run repeatedly:
```

```
}
```

```
//With the settings above, this IRS will trigger each 500ms.
```

```
ISR(TIMESTAMP_vect){
```

```
    TCNT1 = 0; //First, set the timer back to 0 so it resets for next interrupt
```

```
    LED_STATE = !LED_STATE; //Invert LED state
```

```
    digitalWrite(13,LED_STATE); //Write new state to the LED on pin D5
```

```
}
```

- Each time the interrupt is triggered, we go to the **related ISR vector**.
 - Since we have 3 timers we have 6 ISR vectors
 - two for each timer and they have these names:
-

TIMER1_COMPA_vect

TIMER1_COMPB_vect

TIMER2_COMPA_vect

TIMER2_COMPB_vect

TIMER3_COMPA_vect

TIMER4_COMPB_vect

Two ISR at once

- You can combine these interruptions with multiple timers at the same time.
- For example the code below is set to make an interruption each 100ms on timer 1 and each 8.12ms on timer 2.
- Each interruption we invert the pin state of D5 and D6.

```
bool LED_STATE1 = true;
bool LED_STATE2 = true;

void setup() {
  pinMode(5,OUTPUT);
  pinMode(6,OUTPUT);
  cli();                      //stop interrupts for till we make the settings
  //Timer 1 (interrupt each 100ms)
  TCCR1A = 0;                 // Reset entire TCCR1A to 0
  TCCR1B = 0;                 // Reset entire TCCR1B to 0
  TCCR1B |= B00000100;        //Set CS12 to 1 so we get prescalar 256
  TIMSK1 |= B00000010;        //Set OCIE1A to 1 so we enable compare match A
  OCR1A = 6250;               //Finally we set compare register A to this value

  //Timer2 (interrupt each 8.128 ms)
  // interrupt time = 1/(16Mhz/1024) * 127 = 8.128ms;
  TCCR2A = 0;                 // Reset entire TCCR1A to 0
  TCCR2B = 0;                 // Reset entire TCCR1B to 0
  TCCR2B |= B00000111;        //Set CS20, CS21 and CS22 to 1 so we get prescalar 1024
  TIMSK2 |= B00000100;        //Set OCIE1B to 1 so we enable compare match B
  OCR2B = 127;                //Finally we set compare register B to this value
  sei();                      //Enable back the interrupts
}
```

```
void loop() {  
  // put your main code here, to run repeatedly:  
}
```

```
//With the settings above, this IRS will trigger each 8.128ms.
```

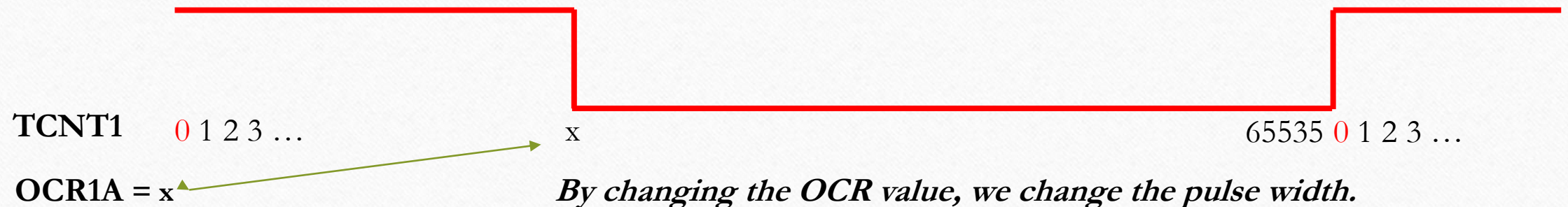
```
ISR(TIMER2_COMPB_vect){  TCNT2 = 0;  
  LED_STATE1 = !LED_STATE1;  //Invert LED state  
  digitalWrite(5,LED_STATE1); //Write new state to the LED on pin D5  
}
```

```
//With the settings above, this IRS will trigger each 100ms.
```

```
ISR(TIMER1_COMPA_vect){  
  TCNT1 = 0;  //First, set the timer back to 0 so it resets for next interrupt  
  LED_STATE2 = !LED_STATE2;  //Invert LED state  
  digitalWrite(6,LED_STATE2); //Write new state to the LED on pin D5  
}
```


PWM with Timer ISR

- Using this same code but without resetting the **TCNT** register when we reach the value, we can **create PWM signals**.
- This is the same process the **analogWrite** is using.
- But in this case we could use any other pin, **not just the PWM pins** of the Arduino.



-
- when each of the OCRA or OCRB is reached, we invert the pulse.
 - By changing the **OCR** value, we change the **pulse width**.
 - The frequency is the same, but the width is changing.

-
- create some PWM signals on pin 2 and 3 and we control the frequency, and the pulse width and best of all the precision.
 - analogWrite has a precision of 8 bits so values from 0 to 255.
 - In this case, you can control the **pulse** with more precisely, with values from 0 to 65535.

Example Code

- we equal OCRA and OCRB to the analogRead from two potentiometers.
- The pulse width could go from 1000 to 2000
 - but you could set it to whatever you want in a range from 0 to 65 thousands in case of timer 1.

```
bool A_STATE = true;
```

```
bool B_STATE = true;
```

```
void setup() {
```

```
  pinMode(2,OUTPUT);
```

```
  pinMode(3,OUTPUT); //LED
```

```
  pinMode(A0, INPUT);
```

```
  pinMode(A1, INPUT); //POT
```

```
  cli(); //stop interrupts for till we make the settings
```

```
  //Timer 1 (interrupt each 50ms)
```

```
  TCCR1A = 0; // Reset entire TCCR1A to 0
```

```
  TCCR1B = 0; // Reset entire TCCR1B to 0
```

```
  TCCR1B |= B00000100; //Set CS12 to 1 so we get prescalar 256
```

```
  TIMSK1 |= B00000110; //Set OCIE1A and OCIE1B to 1 -> compare match A and B
```

```
  sei(); //Enable back the interrupts
```

```
}
```

```
void loop() {  
  OCR1A = map(analogRead(A0),0,1024,1000,2000);  
  OCR1B = map(analogRead(A0),0,1024,1000,2000);  
}
```

```
ISR(TIMESTAMP_vect){  
  A_STATE = !A_STATE;      //Invert LED state  
  digitalWrite(2,A_STATE); //Write new state to the LED on pin D5  
}
```

```
ISR(TIMESTAMP_vect){  
  B_STATE = !B_STATE;      //Invert LED state  
  digitalWrite(3,B_STATE); //Write new state to the LED on pin D5  
}
```